

# SALES ANALYTICS: DATABASE SCHEMA, SAMPLE DATA & QUERY EXAMPLES

## PROJECT OVERVIEW:

- The **Sales Analytics** project is a relational database designed to manage and analyze sales operations. It integrates customer, product, salesperson, and order data to provide actionable insights for business decision-making.
  - The schema demonstrates **SQL fundamentals** (DDL, DML) and **advanced analytics** (aggregations, window functions, ranking).
- 

## OBJECTIVES:

1. **Data Management** – Build a structured relational schema with integrity constraints.
  2. **Business Insights** – Track customer demographics, product pricing, and sales performance.
  3. **Order Analysis** – Identify frequent buyers, high-value products, and sales trends.
  4. **Advanced Analytics** – Apply ranking, partitioning, and window functions for deeper insights.
  5. **Decision Support** – Provide executives with clear, actionable data for strategy and planning.
- 

## DATABASE SCHEMA:

### 1. Customers

- Stores customer details (name, gender, city, email).
- **30 sample customers** across multiple cities.

### 2. Products

- CatLog of items with **name, category, and price**.
- Categories include *Electronics* and *Furniture*.

### 3. Salespersons

- Sales team members with **region and target amount**.

### 4. Orders

- Transaction table linking **customers, products, and salespersons**.
  - Includes **quantity and order date**.
  - **FOREIGN KEYS ENSURE REFERENTIAL INTEGRITY.**
-

## DATA HIGHLIGHTS:

- Customers from **Chennai, Bangalore, Hyderabad, Mumbai, Delhi, Kochi, Pune, etc.**
  - Products range from **Laptop (₹55,000)** to **USB Hub (₹700)**.
  - Salespersons cover **North, South, East, West regions**.
  - Orders simulate real transactions with varying dates and quantities.
- 

## KEY SQL QUERIES:

### Basic Retrieval

- Distinct cities and product categories.
- Customer names with email IDs.
- Products with tax-adjusted prices.

### Filtering

- Customers from Hyderabad.
- Products priced above ₹10,000.
- Orders placed after 2025-01-12.

### Aggregations

- Customer count by city and gender.
- Average product price per category.
- Earliest and latest order dates.

### Advanced Analytics

- **Window Functions:**
    - Total orders per customer.
    - Ranking products by price (global and within category).
  - **HAVING Clause:**
    - Cities with more than 2 customers.
    - Categories with average price > ₹5,000.
-

## INSIGHTS ENABLED:

- **Top-selling products** and their demand.
  - **Customer distribution** across regions.
  - **Salesperson performance** vs. targets.
  - **Order timelines** for trend analysis.
  - **Category-level profitability.**
- 

## SQL QUERIES AND EXECUTION:

The are the SQL queries in this project

```
0      -- 1 .distinct cities
1 •      select
2          distinct city
3      from customers;
4
5      -- 2 .Retrieve distinct product categories from the Products table.
6 •      select
7          distinct category
8      from products;
9
10     -- 3.Display customer names and their email IDs renamed as Customer_Name and Email_ID.
11 •      select
12          customername as customer_name,
13          email as email_id
14      from customers;
15
16     -- 4.Show product name, price, and (Price x 2) as DoublePrice.
17 •      select
18          productname,price,(price*2) as double_price
19      from products;
20
21     -- 5.Show product name and price after adding a 10% tax.
22 •      select
23          productname ,price,( price+(price*0.1)) as price_with_tax
24      from products;
25
26     -- 6.Find all customers who live in Hyderabad.
27 •      SELECT CustomerName
28      FROM Customers
29      WHERE City = 'Hyderabad';
30
```

```
161  -- 7.Show all products priced above 10,000.
162  •  select productname
163      from products
164      where price >10000;
165
166  -- 8. List all orders placed after 2025-01-12.
167  •  select orderid
168      from orders
169      where orderdate > '2025-01-12';
170
171  -- 9. Find products whose price is NOT equal to 1500.
172
173  •  select productname
174      from products
175      where price <> 1500;
176
177  -- 10. Find customers whose Email is NULL.
178  •  select customername
179      from customers
180      where email is null;
181
182  -- 11.List all orders where quantity is NOT NULL.
183  •  select orderid
184      from orders
185      where quantity is not null;
186
187  -- 12.List the female customers who live in Chennai.
188  •  select customername
189      from customers
190      where gender='female';
```

```

!92 • SELECT CustomerName, City
!93 FROM Customers
!94 WHERE City in ('Chennai', 'Bangalore', 'Hyderabad');
!95
!96 -- 14.Show products whose category is NOT IN ('Electronics', 'Furniture')
!97 • SELECT productname,category
!98 FROM products
!99 WHERE category not in ('Electronics', 'Furniture');
!100
!101 -- ORDER BY & LIMIT
!102 -- 1.List all customers ordered by their Customer Name in ascending order.
!103 • select customerid ,customername
!104 from customers
!105 order by customername;
!106
!107 -- 2.Display the top 3 products by price.
!108 • select productname,price
!109 from products
!110 order by price desc
!111 limit 3;
!112
!113 -- 3.List the top 3 most expensive products whose price is > 5,000, ordered by price DESC.
!114 • SELECT ProductName, Price
!115 FROM Products
!116 WHERE Price > 5000
!117 ORDER BY Price DESC
!118 LIMIT 3;
!119
!120 -- 4.Show the number of customers in each city and order the result by customer count DESC.
!121 • SELECT City, COUNT(CustomerID) AS CustomerCount
!122 FROM Customers
!123 GROUP BY City
!124 ORDER BY CustomerCount DESC;
!125

```

---

```
226 -- 5.Retrieve customers in ('Chennai', 'Pune', 'Hyderabad') and sort by name.
227 • select city,customername
228 from customers
229 where city in ('Chennai', 'Pune', 'Hyderabad')
230 ORDER BY customername ;
231
232 -- 6.Retrieve customers in given cities, display City + CustomerName, sorted by city and name.
233 • select city,customername
234 from customers
235 where city in ('Chennai', 'Pune', 'Hyderabad')
236 ORDER BY customername,city ;
237
238 -- 7.List customers whose name starts with 'A' and sort by CustomerID.
239 • select customername
240 from customers
241 where customername like('a%');
242
243 -- 8.Find the total number of customers in the Customers table.
244 • select
245 count(*) as number_of_customers
246 from customers;
247
248 -- 9.Show the number of customers in each city.
249 • select city,
250 count(customername) as number_of_customers
251 from customers
252 group by city;
253
254 -- 10. Show the total number of customers in each gender group.
255 • SELECT Gender,
256 COUNT(CustomerName) AS Number_of_Customers
257 FROM Customers
258 GROUP BY Gender;
```

```

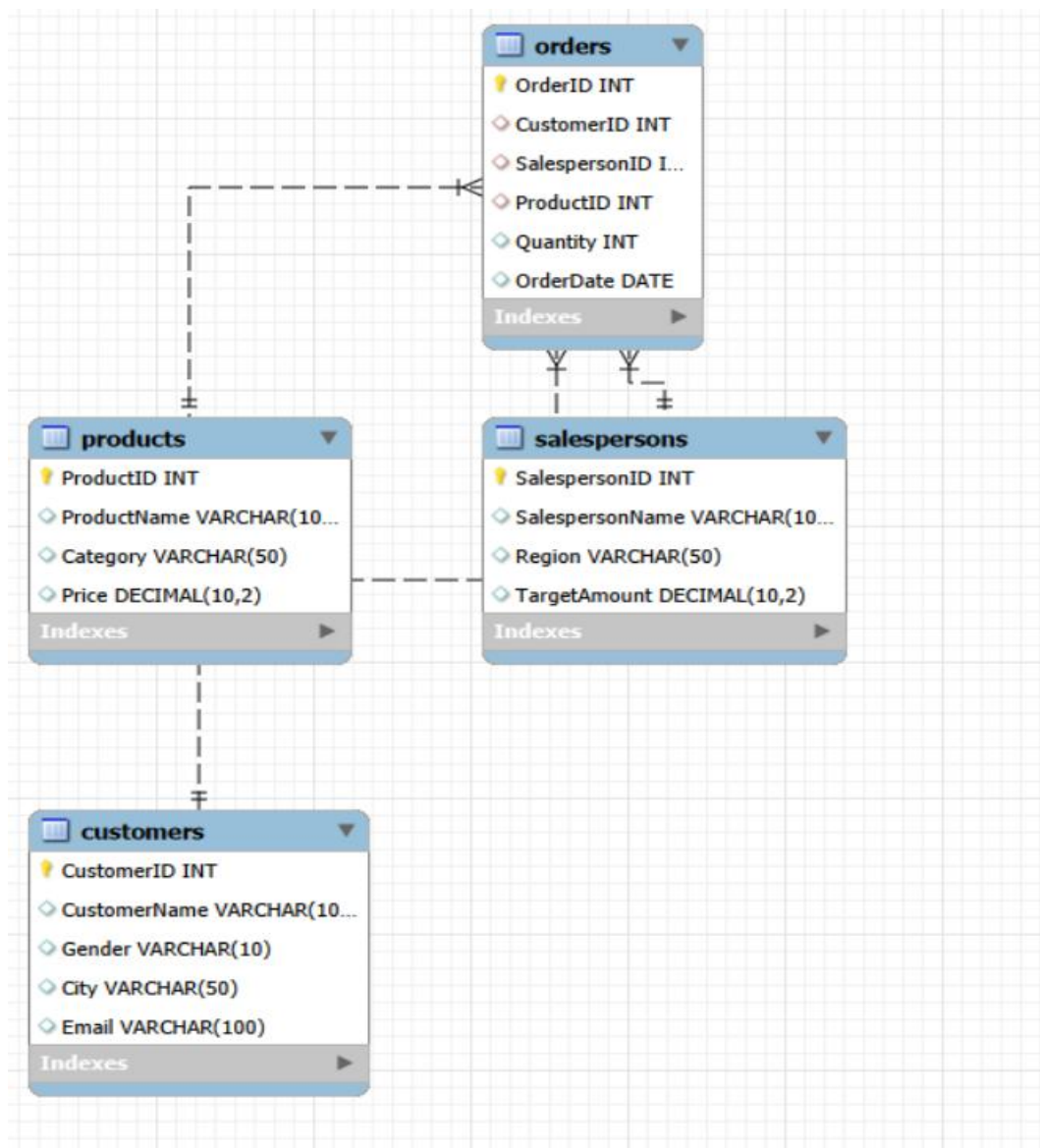
291  -- Display each order with the total number of orders placed by that customer
292  • SELECT
293      OrderID,
294      CustomerID,
295      COUNT(OrderID) OVER (PARTITION BY CustomerID) AS TotalOrdersByCustomer
296  FROM Orders ;
297
298  -- 2.  Show each product with its price and the average price of all products.
299  • select
300      productname,
301      price,
302      avg(price) over(partition by productname)as avarage_price_of_product
303  from products;
304
305  -- 3.  Rank all products based on price from highest to lowest using a window function.
306  • SELECT
307      ProductName,
308      Price,
309      RANK() OVER (ORDER BY Price DESC) AS PriceRank
310  FROM Products;
311  -- Show product price ranking within each category using PARTITION BY
312  • SELECT
313      Category,
314      ProductName,
315      Price,
316      RANK() OVER ( partition by category order  BY Price DESC) AS PriceRank
317  FROM Products;
318

```

---

## ENHANCED ENTITY–RELATIONSHIP DIAGRAM:

This image shows the relationships between the tables in the database schema.



---

## CONCLUSION:

The **Sales Analytics project** demonstrates how SQL can be applied to design a structured database, manage sales data, and generate meaningful insights. It highlights customer demographics, product pricing, and order trends while showcasing advanced techniques like grouping, ranking, and window functions. Overall, the project proves SQL's value in turning raw data into **actionable business intelligence**.

---